# OpenEMR Integration

## 1. Overview

### 1.1 Description

**OpenEMR Integration** connects **OpenEMR** (open-source electronic health records and medical practice management system) with **Newo Platform**.

It enables:

- Checking available appointment slots for a selected provider and facility
- Booking appointments with automatic patient record search and creation
- Cancelling existing appointments
- Pre-configured facility and provider selection during setup
- OAuth2 token management with automatic refresh on expiration

This integration is designed for **medical practices and clinics using OpenEMR** who need **automated appointment scheduling through a conversational AI agent** (voice or chat).

### 1.2 Use Cases

- When a patient asks about available slots -> AI checks the configured provider's schedule and returns open time slots
- When a patient wants to book an appointment -> Search or create patient record, then create appointment in OpenEMR
- When a patient wants to cancel an appointment -> Delete the appointment via OpenEMR API
- When a new patient books -> Automatically create patient record in OpenEMR before booking
- When the project is published -> Facilities and providers are fetched and stored for selection

### 1.3 Supported Features

| Feature | Supported | Notes |
|---|---|---|
| Check Availability | Yes | Date range lookup for selected provider and facility |
| Book Appointment | Yes | With automatic patient creation if needed |
| Cancel Appointment | Yes | Requires booking_id and contact_id from prior booking |
| Facility Selection | Yes | Pre-configured during setup via enum attribute |
| Provider Selection | Yes | Pre-configured during setup, filtered by facility |
| Patient Search | Yes | By phone number via OpenEMR API |
| Patient Creation | Yes | Auto-creates during booking flow |
| Token Auto-Refresh | Yes | OAuth2 with password grant and refresh token rotation |
| Multi-Provider Scheduling | No | Single provider selected during setup |
| Reschedule Appointment | No | Cancel + rebook as workaround |
| Historical Data Import | No | Only new events after activation |

## 2. Requirements

Before installation:

- Active **OpenEMR** instance with REST API enabled
- **OAuth2 Client ID** and **Client Secret** registered in OpenEMR
- **Username**, **password**, and **email** for an OpenEMR user with API access
- The OpenEMR instance must be accessible via HTTPS from the platform

## 3. How to Setup

### 3.1 Step 1 — Get API Credentials

1. Log in to your **OpenEMR** instance as an administrator
2. Navigate to **Administration -> System -> API Clients**
3. Register a new OAuth2 client application
4. Note the **Client ID** and **Client Secret**
5. Ensure the API user has appropriate permissions for appointments and patient management

### 3.2 Step 2 — Connect in Platform

1. Open **Newo -> Projects**
2. Set the following attributes in **Builder / Attributes**:

| Attribute | Required | Description |
|---|---|---|
| `openemr_base_url` | Yes | Base URL of your OpenEMR instance (e.g., `https://emr.example.com`) |
| `openemr_client_id` | Yes | OAuth2 Client ID |
| `openemr_client_secret` | Yes | OAuth2 Client Secret |
| `openemr_username` | Yes | OpenEMR user username for API access |
| `openemr_password` | Yes | OpenEMR user password |
| `openemr_email` | Yes | OpenEMR user email |
| `openemr_scope` | No | OAuth2 scopes (pre-configured with full access) |
| `openemr_facility` | No | Selected facility (auto-populated during setup) |
| `openemr_provider` | No | Selected provider (auto-populated during setup) |
| `openemr_enable_slot_check` | No | Enable availability checks (default: `True`) |
| `openemr_enable_booking` | No | Enable appointment creation (default: `True`) |
| `openemr_enable_cancellation` | No | Enable cancellations (default: `True`) |
| `openemr_check_existing_client` | No | Check if patient exists before creating (default: `True`) |
| `openemr_show_for_days` | No | Number of days to check for availability |

| | | |
|---|---|---|
| `openemr_override_agent_attributes` | No | Override SuperAgent attribute schemas (default: `True`) |

3. Click **Save + Publish All**
4. On publish, the SetupFlow automatically:
    - Exchanges credentials for an OAuth2 access token (password grant)
    - Creates HTTP connectors for API and token operations
    - Fetches all facilities and populates the facility selector
    - Fetches providers for the selected facility and populates the provider selector
    - Registers availability, booking, and cancellation tools with the agent
    - Injects payload schemas for the ConvoAgent tool calling framework

# 4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

## 4.1 How the Integration Works

The integration uses OpenEMR's REST API for appointment management, patient records, and facility data. Unlike multi-provider integrations, the facility and provider are pre-selected during setup, simplifying runtime slot lookups. The OAuth2 token is auto-refreshed on 401 responses using the refresh token grant.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in OpenEMR

**Available Triggers**

| Trigger | Event | Description |
|---|---|---|
| **Check Availability** | `get_available_slots` | When the agent needs to look up open slots |
| **Book Appointment** | `book_slot` | When the agent confirms a booking |
| **Cancel Appointment** | `convo_agent_cancel_booking` | When the agent processes a cancellation |
| **Setup** | `project_publish_finish` | Initializes integration on deploy |

**Available Actions**

- **Get Facilities** — Fetch all clinic facilities (GET `/apis/default/api/facility`)
- **Get Providers** — Fetch providers at a facility (GET `/apis/default/api/user`)
- **Get Slots** — Fetch available appointment slots (GET `/apis/default/api/available_slots`)
- **Search Patient** — Find existing patient by phone (GET `/apis/default/api/patient_by_phone`)
- **Create Patient** — Create a new patient record (POST `/apis/default/api/patient`)
- **Create Appointment** — Book an appointment slot (POST `/apis/default/api/patient/{id}/appointment`)
- **Cancel Appointment** — Delete an existing appointment (DELETE `/apis/default/api/patient/{id}/appointment/{appt_id}`)
- **Get Token** — Exchange credentials for OAuth2 token (POST `/oauth2/default/token`)

## 4.2 Architecture Diagrams

**Overall Sequence**

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as OpenEMR Integration
    participant API as OpenEMR API

    Note over I,API: Setup (on publish)
    A->>I: project_publish_finish
    I->>API: POST /oauth2/default/token (password grant)
    API-->>I: access_token + refresh_token
    I->>API: GET /facility
    API-->>I: Facilities list
    I->>API: GET /user?facility_id={id}
    API-->>I: Providers list
    I->>A: Tools registered, enums populated

    Note over U,API: Availability Check
    U->>A: "Any slots on Monday?"
    A->>I: get_available_slots (date, time)
    I->>API: GET /available_slots?date_from=...&date_to=...&aid=...&fid=...
    API-->>I: Available slots by date
    I->>A: result_success (availability[])
    A->>U: "Here are the available times..."

    Note over U,API: Booking
    U->>A: "Book 2:00 PM please"
    A->>I: book_slot (customerInfo, bookingDateTime)
    I->>API: GET /patient_by_phone?phone=...
    API-->>I: Patient results
    alt Patient not found
        I->>API: POST /patient (fname, lname, DOB, sex, phone)
        API-->>I: New patient_id
    end
    I->>API: POST /patient/{id}/appointment
    API-->>I: appointment_id
    I->>A: result_success (booking_id)
    A->>U: "Your appointment is confirmed!"

    Note over U,API: Cancellation
    U->>A: "Cancel my appointment"
    A->>I: convo_agent_cancel_booking (booking_id, contact_id)
    I->>API: DELETE /patient/{contact_id}/appointment/{booking_id}
    API-->>I: record deleted
    I->>A: result_success (cancelled)
    A->>U: "Your appointment has been cancelled"
```

**AvailabilityFlow**

```
flowchart TD
    E["get_available_slots"] --> CHK{"Slot check<br>enabled?"}
```

```
    CHK -->|"No"| SKIP["Return (disabled)"]
    CHK -->|"Yes"| PARSE["Parse payload:<br>date, time"]
    PARSE --> ATTR["Read attributes:<br>base_url, facility,<br>provider,
show_for_days"]
    ATTR --> CALC["Calculate date range:<br>date_from = requested date<br>date_to =
date_from + show_for_days"]
    CALC --> API["GET /available_slots<br>?date_from=...&date_to=...<br>&aid=
{provider_id}<br>&fid={facility_id}"]
    API --> OK{Success?}
    OK -->|"No"| ERR["result_error"]
    OK -->|"Yes"| FMT["Format slots:<br>group by date,<br>convert to HH:MM AM/PM"]
    FMT --> OUT["result_success<br>{availability[]}"]
```

**BookingFlow**

```
flowchart TD
    E["book_slot"] --> CHK{"Booking<br>enabled?"}
    CHK -->|"No"| SKIP["Return (disabled)"]
    CHK -->|"Yes"| PHONE["Extract phone from<br>customerInfo.phoneNumber"]
    PHONE --> SEARCH["GET /patient_by_phone<br>?phone={phoneNumber}"]
    SEARCH --> FOUND{"Patient<br>found?"}
    FOUND -->|"Yes"| USE["Use existing<br>patient ID"]
    FOUND -->|"No"| CREATE["POST /patient<br>{fname, lname, DOB,<br>sex,
phone_contact}"]
    CREATE --> NEW["Extract new<br>patient ID (pid)"]
    USE --> BOOK["POST /patient/{id}/appointment<br>{pc_eventDate, pc_startTime,
<br>pc_endTime (+15min),<br>pc_facility, pc_aid,<br>pc_title: Office Visit}"]
    NEW --> BOOK
    BOOK --> OK{"Success?"}
    OK -->|"No"| ERR["result_error"]
    OK -->|"Yes"| OUT["result_success<br>{booking_id, contact_id}"]
```

**CancellationFlow**

```
flowchart TD
    E["convo_agent_cancel_booking"] --> CHK{"Cancellation<br>enabled?"}
    CHK -->|"No"| SKIP["Return (disabled)"]
    CHK -->|"Yes"| VAL{"booking_id and<br>contact_id present?"}
    VAL -->|"No"| ERR["result_error:<br>missing IDs"]
    VAL -->|"Yes"| API["DELETE /patient/{contact_id}<br>/appointment/{booking_id}"]
    API --> OK{"Response:<br>record deleted?"}
    OK -->|"No"| ERR2["result_error"]
    OK -->|"Yes"| OUT["result_success<br>{cancel_booking}"]
```

**UtilsFlow (Token Management)**

```
flowchart TD
    REQ["Any API request"] --> TOK{"Access token<br>available?"}
    TOK -->|"No"| REFRESH["POST /oauth2/default/token"]
```

```
    TOK -->|"Yes"| HTTP["Send via<br>openemr_connector"]
    HTTP --> STATUS{"HTTP<br>status?"}
    STATUS -->|"200/201"| CHECK{"Response has<br>error field?"}
    CHECK -->|"No"| SUCCESS["openemr_result_success"]
    CHECK -->|"Yes"| ERR["openemr_result_error"]
    STATUS -->|"401"| REFRESH
    REFRESH --> GRANT{"refresh_token<br>available?"}
    GRANT -->|"Yes"| REF_GRANT["grant_type: refresh_token"]
    GRANT -->|"No"| PWD_GRANT["grant_type: password<br>(username + password +
email)"]
    REF_GRANT --> TOK_OK{"New token?"}
    PWD_GRANT --> TOK_OK
    TOK_OK -->|"Yes"| SAVE["Save access_token +<br>refresh_token,<br>retry request
(max 3)"]
    TOK_OK -->|"No"| ERR
    STATUS -->|"4xx/5xx"| ERR
```

### 4.3 Where to Test

Testing can be done through the **Newo conversation interface** (voice or chat). Each flow (Availability, Booking, Cancellation) has a dedicated webhook test endpoint for connectivity validation.

### 4.4 Testing and Verification

**To test the integration:**

1. **Setup**: Publish the project and verify token exchange succeeds (check `openemr_access_token` is populated)
2. **Facilities**: After publish, verify `openemr_facility` enum is populated with your clinics
3. **Providers**: Verify `openemr_provider` enum shows providers for the selected facility
4. **Availability**: Ask "What slots are available on Monday?" — verify slots returned with correct times
5. **Booking**: Complete a booking conversation — verify appointment appears in OpenEMR calendar
6. **Cancellation**: Request cancellation of a booked appointment — verify appointment removed from OpenEMR

**If no action occurs:**

- Ensure `openemr_base_url`, `openemr_client_id`, and `openemr_client_secret` are set correctly
- Verify `openemr_username`, `openemr_password`, and `openemr_email` are valid OpenEMR credentials
- Confirm `openemr_access_token` was obtained (check it's not empty after setup)
- Ensure the relevant feature flags are enabled ( `openemr_enable_booking`, `openemr_enable_slot_check`, `openemr_enable_cancellation` )
- Verify `openemr_facility` and `openemr_provider` were populated during setup
- Check that OpenEMR's REST API is enabled and accessible from the platform
- Re-publish the project if credentials were changed

> *Note: This integration performs real operations in OpenEMR. Appointments created or cancelled through the agent are reflected in the live OpenEMR system.*

## 5. FAQ

**Q: How does facility and provider selection work?** A: Unlike multi-provider integrations, OpenEMR uses a pre-selection model. During setup, all facilities are fetched and presented as an enum dropdown. After selecting a facility, providers for that facility are fetched and presented similarly. These selections are stored and used for all subsequent availability checks and bookings.

**Q: What patient information is required for booking?** A: First name, last name, date of birth (YYYY-MM-DD), sex (Male/Female), and phone number are required. Middle name is optional. If the phone number is not available, the agent will request it before proceeding.

**Q: What happens if the OAuth token expires?** A: The UtilsFlow automatically detects 401 responses. If a refresh token is available, it uses the `refresh_token` grant to obtain a new access token. If no refresh token exists, it falls back to the `password` grant using the stored credentials. The original request is retried up to 3 times.

**Q: How many days of availability does the agent check?** A: Configurable via the `openemr_show_for_days` attribute. The API call checks from the requested date through the configured number of days.

**Q: What is the default appointment duration?** A: 15 minutes. All appointments are created with `pc_duration: 15` and the end time is calculated as start time + 15 minutes. The appointment title defaults to "Office Visit".

**Q: Does the integration support multiple facilities simultaneously?** A: No. A single facility and provider are selected during setup. To switch facilities, update the `openemr_facility` attribute and re-publish the project to refresh the provider list.

**Q: What OAuth2 scopes are required?** A: The integration pre-configures a comprehensive scope set covering appointments, patients, practitioners, locations, and related resources. The scope attribute is auto-populated during setup and typically does not need manual configuration.